

# HCI & CG LAB TASK

**Name:** Raza e Mustafa

**Roll No:** 2k24/CSE/122

**Submitted to:** Sir Rajesh Kumar

## Task 1: Display Pixel Density (PPI/DPI) Calculator

```
import math

print("--- TASK 1: DISPLAY PIXEL DENSITY  
CALCULATOR ---")

# 1. Inputs
w_px = int(input("Enter horizontal resolution  
(pixels): "))
h_px = int(input("Enter vertical resolution  
(pixels): "))
d_inches = float(input("Enter physical diagonal  
size (inches): "))

# 2. Calculations
total_pixels = w_px * h_px
gcd_val = math.gcd(w_px, h_px)
aspect_w = w_px // gcd_val
aspect_h = h_px // gcd_val

# Diagonal pixels  $P_d = \sqrt{W^2 + H^2}$ 
p_d = math.sqrt(w_px**2 + h_px**2)
dpi = p_d / d_inches

# 3. Classification
if dpi < 100:
```

```

        category = "Low Density (Standard Monitor)"
elif 100 <= dpi <= 200:
    category = "Medium Density (HD Display)"
else:
    category = "High Density (Retina / Mobile)"

# Display Results
print("\n--- DISPLAY METRICS ANALYSIS ---")
print(f"Total Pixel Count: {total_pixels:,} pixels")
print(f"Aspect Ratio: {aspect_w}:{aspect_h}")
print(f"Calculated DPI: {dpi:.2f} DPI")
print(f"Density Category: {category}\n")

```

Screenshot 1 (Desktop):

The screenshot shows a Python IDE with a dark theme. The editor displays a script with density classification logic. The output window shows the results of running the script.

```

# Density classification
23 if dpi < 100:
24     density = "Low Density (Standard Monitor)"
25 elif dpi <= 200:
26     density = "Medium Density (HD Display)"
27 else:
28     density = "High Density (Retina / Mobile)"
29
30 # Output
31 print("\n--- DISPLAY METRICS ANALYSIS ---")
32 print(f"Total Pixel Count : {total_pixels:,} pixels")
33 print(f"Aspect Ratio : {aspect_width}:{aspect_height}")
34 print(f"Calculated DPI : {dpi:.2f} DPI")

```

Output:

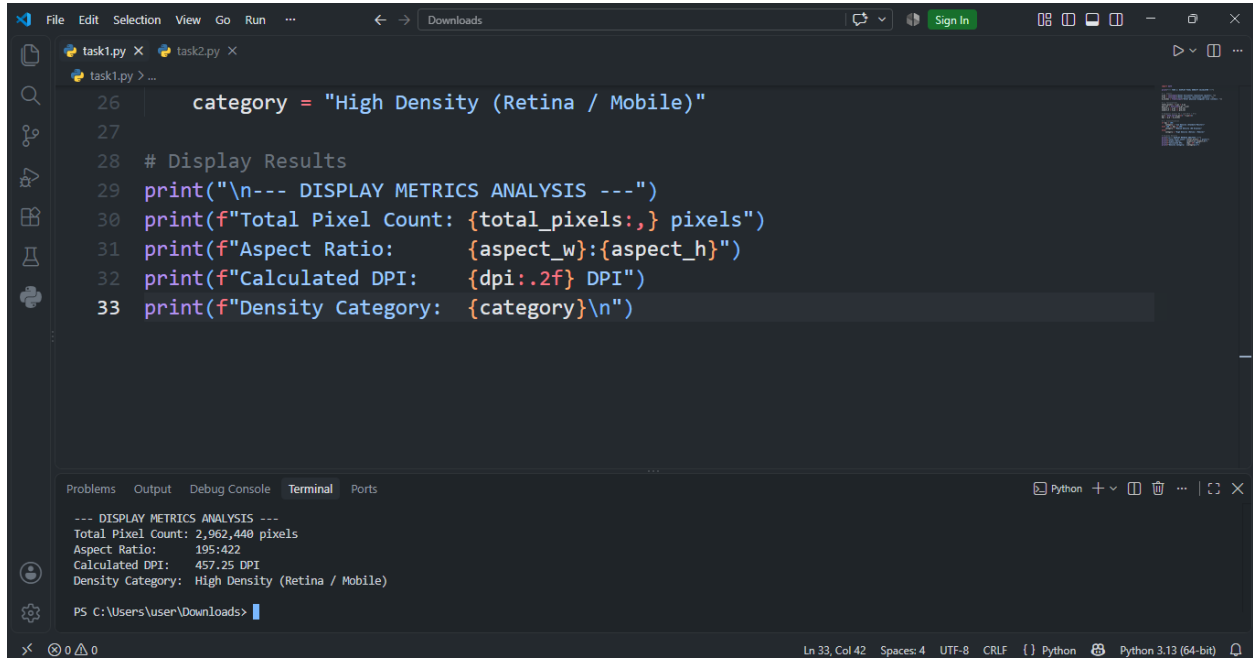
```

--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count : 2,073,600 pixels
Aspect Ratio : 16:9
Calculated DPI : 91.79 DPI
Density Category : Low Density (Standard Monitor)
PS C:\Users\user\Downloads>

```

The status bar at the bottom indicates the file is at line 35, column 39, with 4 spaces, UTF-8 encoding, CRLF line endings, and is running Python 3.13 (64-bit).

## Screenshot 2 (Smartphone):



The screenshot shows a Visual Studio Code editor window with a Python script named `task1.py` open. The script contains the following code:

```
26 category = "High Density (Retina / Mobile)"
27
28 # Display Results
29 print("\n--- DISPLAY METRICS ANALYSIS ---")
30 print(f"Total Pixel Count: {total_pixels:,} pixels")
31 print(f"Aspect Ratio:      {aspect_w}:{aspect_h}")
32 print(f"Calculated DPI:     {dpi:.2f} DPI")
33 print(f"Density Category:  {category}\n")
```

The bottom panel of the editor shows the `Terminal` tab, which displays the output of the script:

```
--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count: 2,962,440 pixels
Aspect Ratio:      195:422
Calculated DPI:     457.25 DPI
Density Category:  High Density (Retina / Mobile)
```

The terminal prompt is `PS C:\Users\User\Downloads>`. The status bar at the bottom indicates the file is at line 33, column 42, using UTF-8 encoding, and the Python interpreter is set to Python 3.13 (64-bit).

## Task 2: Environment Setup & Synthetic Image Matrix Creation

```
import math

# Input
Wpx = int(input("Enter horizontal resolution (pixels): "))
Hpx = int(input("Enter vertical resolution (pixels): "))
Dinches = float(input("Enter physical diagonal size (inches): "))

# Total pixel count
total_pixels = Wpx * Hpx

# Simplified aspect ratio
gcd = math.gcd(Wpx, Hpx)
aspect_width = Wpx // gcd
aspect_height = Hpx // gcd

# Calculate diagonal pixels
Pd = math.sqrt(Wpx**2 + Hpx**2)

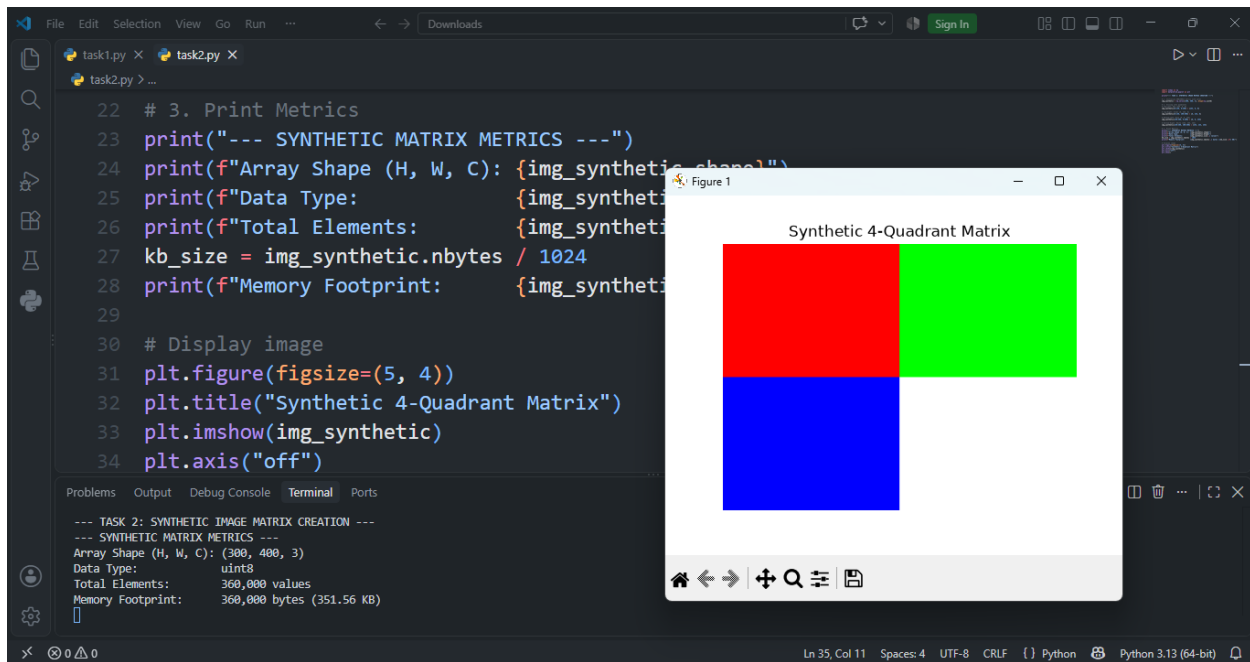
# Calculate DPI/PPI
dpi = Pd / Dinches

# Density classification
```

```
if dpi < 100:
    density = "Low Density (Standard Monitor)"
elif dpi <= 200:
    density = "Medium Density (HD Display)"
else:
    density = "High Density (Retina / Mobile)"

# Output
print("\n--- DISPLAY METRICS ANALYSIS ---")
print(f"Total Pixel Count : {total_pixels:,} pixels")
print(f"Aspect Ratio : {aspect_width}:{aspect_height}")
print(f"Calculated DPI : {dpi:.2f} DPI")
print(f"Density Category : {density}")
```

## Task 2 Output:



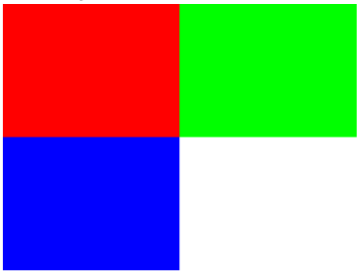
```
22 # 3. Print Metrics
23 print("--- SYNTHETIC MATRIX METRICS ---")
24 print(f"Array Shape (H, W, C): {img_synthetic.shape}")
25 print(f>Data Type: {img_synthetic.dtype}")
26 print(f>Total Elements: {img_synthetic.size}")
27 kb_size = img_synthetic.nbytes / 1024
28 print(f"Memory Footprint: {kb_size} KB")
29
30 # Display image
31 plt.figure(figsize=(5, 4))
32 plt.title("Synthetic 4-Quadrant Matrix")
33 plt.imshow(img_synthetic)
34 plt.axis("off")
```

Problems Output Debug Console Terminal Ports

```
--- TASK 2: SYNTHETIC IMAGE MATRIX CREATION ---
--- SYNTHETIC MATRIX METRICS ---
Array Shape (H, W, C): (300, 400, 3)
Data Type: uint8
Total Elements: 360,000 values
Memory Footprint: 360,000 bytes (351.56 KB)
```

Figure 1

Synthetic 4-Quadrant Matrix



## Task 3: Channel Slicing & Isolation

```
import os
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

print("--- TASK 3: CHANNEL SLICING & ISOLATION ---")

image_path = "sample.jpg"
```

```

# Agar sample.jpg maujood na ho to yahin create
kar lega
if not os.path.exists(image_path):
    temp_grid = np.zeros((300, 400, 3),
dtype=np.uint8)
    temp_grid[0:150, 0:200] = [255, 0,
0]          # Red quadrant
    temp_grid[0:150, 200:400] = [0, 255,
0]          # Green quadrant
    temp_grid[150:300, 0:200] = [0, 0,
255]        # Blue quadrant
    temp_grid[150:300, 200:400] = [255, 255,
255]# White quadrant
    Image.fromarray(temp_grid).save(image_path)

# 1. Load image and convert to RGB array
raw_img = Image.open(image_path).convert("RGB")
img = np.array(raw_img, dtype=np.uint8)

# 2. Extract 2D intensity grids (Axis 2
slicing)
red_2d = img[:, :, 0]
green_2d = img[:, :, 1]
blue_2d = img[:, :, 2]

# 3. Construct isolated 3D color images

```



```
red_only = np.zeros_like(img)
red_only[:, :, 0] = red_2d

green_only = np.zeros_like(img)
green_only[:, :, 1] = green_2d

blue_only = np.zeros_like(img)
blue_only[:, :, 2] = blue_2d

# Print summary
print("--- CHANNEL EXTRACTION SUMMARY ---")
print(f"Original Image Shape: {img.shape}")
print(f"Red Channel 2D Shape: {red_2d.shape} | Mean Intensity: {red_2d.mean():.2f}")
print(f"Green Channel 2D Shape: {green_2d.shape} | Mean Intensity: {green_2d.mean():.2f}")
print(f"Blue Channel 2D Shape: {blue_2d.shape} | Mean Intensity: {blue_2d.mean():.2f}")

# 4. Display 2x3 Subplot Grid
fig, axes = plt.subplots(2, 3, figsize=(12, 8))

# Top Row: 3D Color isolations
axes[0, 0].imshow(red_only)
axes[0, 0].set_title("Red-Only View")
axes[0, 1].imshow(green_only)
```

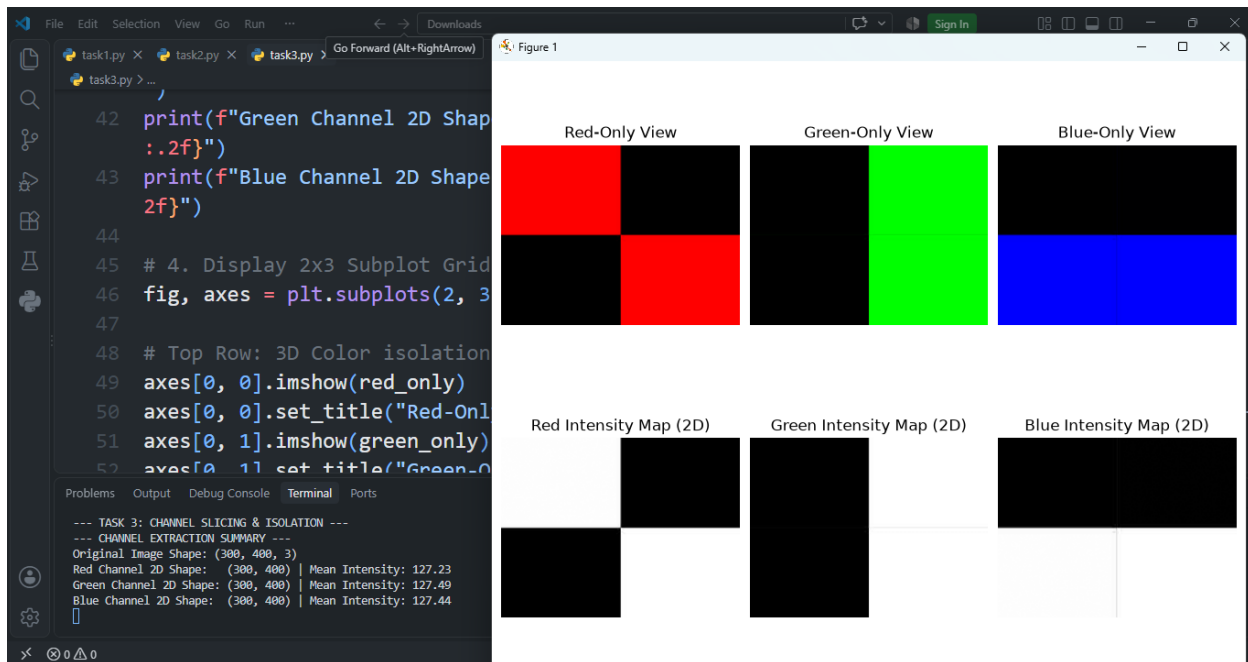
```
axes[0, 1].set_title("Green-Only View")
axes[0, 2].imshow(blue_only)
axes[0, 2].set_title("Blue-Only View")

# Bottom Row: 2D Grayscale intensity maps
axes[1, 0].imshow(red_2d, cmap="gray")
axes[1, 0].set_title("Red Intensity Map (2D)")
axes[1, 1].imshow(green_2d, cmap="gray")
axes[1, 1].set_title("Green Intensity Map (2D)")
axes[1, 2].imshow(blue_2d, cmap="gray")
axes[1, 2].set_title("Blue Intensity Map (2D)")

for ax in axes.flat:
    ax.axis("off")

plt.tight_layout()
plt.show()
```

### Task 3 Output:



### Task 4: Spatial Downsampling & Pixelation via Striding

```
import os  
import numpy as np  
import matplotlib.pyplot as plt  
from PIL import Image  
  
print("--- TASK 4: SPATIAL DOWNSAMPLING &  
PIXELATION ---")  
  
image_path = "sample.jpg"
```

```
# Agar sample.jpg nahi hai toh auto-generate
kar lega
if not os.path.exists(image_path):
    temp = np.zeros((300, 400, 3),
dtype=np.uint8)
    temp[0:150, 0:200] = [255, 0, 0]
    temp[0:150, 200:400] = [0, 255, 0]
    temp[150:300, 0:200] = [0, 0, 255]
    temp[150:300, 200:400] = [255, 255, 255]
    Image.fromarray(temp).save(image_path)

# 1. Load image into numpy array
img =
np.array(Image.open(image_path).convert("RGB"),
dtype=np.uint8)

N = 8

# 2. Downsample taking every N-th pixel (slice
step)
downsampled = img[::N, ::N, :]

# 3. Re-expand back using np.repeat across axes
0 and 1
re_expanded = np.repeat(np.repeat(downsampled,
N, axis=0), N, axis=1)
```

```
# Ensure dimensions match exactly with original
image
re_expanded = re_expanded[:img.shape[0],
:img.shape[1], :]

# 4. Calculate reductions and memory savings
orig_bytes = img.nbytes
down_bytes = downsampled.nbytes
dim_reduction = (1 - (1 / N)) * 100
mem_savings = ((orig_bytes - down_bytes) /
orig_bytes) * 100

print(f"--- DOWNSAMPLING ANALYSIS (N={N}) ---")
print(f"Original Shape:      {img.shape} |
Memory: {orig_bytes:,} bytes")
print(f"Downsampled
Shape:    {downsampled.shape} | Memory:
{down_bytes:,} bytes")
print(f"Re-expanded
Shape:    {re_expanded.shape} | Visual: Blocky
Pixelation")
print(f"Dimension Reduction:
{dim_reduction:.2f}% reduction per axis")
print(f"Memory Savings:      {mem_savings:.2f}%
data reduction")
```

```
# 5. Display visual output
fig, axes = plt.subplots(1, 3, figsize=(14, 5))

axes[0].imshow(img)
axes[0].set_title("Original Image")

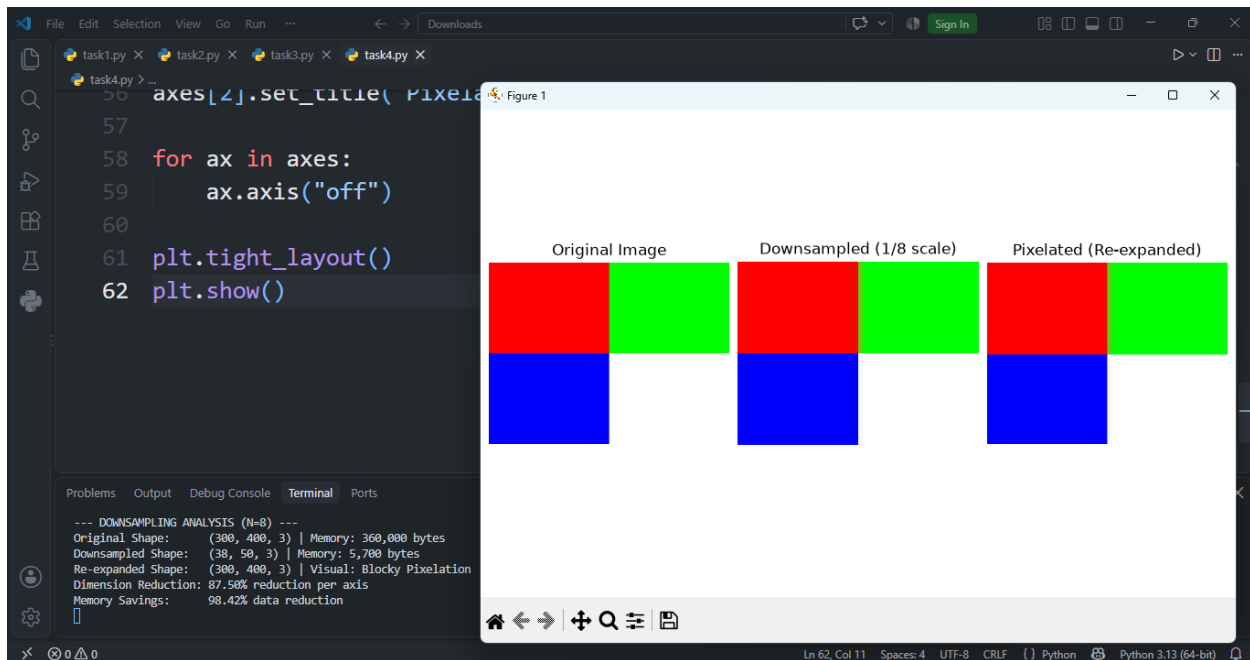
axes[1].imshow(downsampled)
axes[1].set_title(f"Downsampled (1/{N} scale)")

axes[2].imshow(re_expanded)
axes[2].set_title("Pixelated (Re-expanded)")

for ax in axes:
    ax.axis("off")

plt.tight_layout()
plt.show()
```

## Task 4 Output:



**Question:** Why does adding an Alpha channel (RGBA) increase an image array's memory consumption by 33% compared to standard RGB?

**Answer:** A standard RGB image uses **3 channels** per pixel (Red, Green, Blue). Adding an Alpha channel introduces a **4th channel** for transparency, raising the per-pixel requirement from 3 values to 4 values.